МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №7

Тема: «functools та operator»

Варіант 15

Виконала:

Студентка 2 курсу групи ФЕП-22

Сулятицька Христина

Перевірив:

Аспірант Остріков Данило Андрійович

Львів – 2026

Мета роботи.

Ознайомитися з можливостями модулів functools та operator, навчитися створювати функціональні конвеєри обробки даних, застосовувати часткове застосування функцій, згортку reduce, поліморфне завантаження даних через singledispatch і реалізовувати повний порядок порівняння об’єктів за допомогою total_ordering. Для варіанта 15 розробити програму обробки результатів лабораторних робіт студентів.

Теоретичні відомості:

functools.partial фіксує частину аргументів функції та створює нову функцію з простішим інтерфейсом. У роботі partial використано для налаштування етапів map, filter, сортування, вибору Top-N та обчислення підсумкового бала.

functools.reduce послідовно передає накопичене значення та поточний елемент функції-акумулятору. На цій основі реалізовано універсальну функцію pipeline(value, *funcs), яка виконує послідовність перетворень.

module operator надає готові функції itemgetter, attrgetter і methodcaller. Вони дають змогу отримувати поля словників та атрибути об’єктів і викликати методи без додаткових lambda-виразів.

Декоратор singledispatch забезпечує вибір реалізації функції залежно від типу першого аргументу. У програмі load_results підтримує JSON-рядок, шлях str, pathlib.Path, відкритий текстовий файл і готовий список словників; файлові джерела можуть містити JSON або CSV.

Декоратор total_ordering генерує відсутні оператори порівняння на основі __eq__ та __lt__. Клас LabResult визначає порядок за балом, кількістю спроб і іменем студента.

Хід роботи

1. Аналіз предметної області та структури даних

Для варіанта 15 я обрала предметну область «Результати лабораторних робіт». Кожен запис містить поля student (ім’я студента), lab (номер лабораторної), score (отриманий бал), attempts (кількість спроб) і submitted (ознака здачі). Під час обробки програма нормалізує типи даних, очищає ім’я, залишає лише здані роботи та формує рейтинг.

2. Реалізація поліморфного завантаження даних

У файлі loaders.py реалізовано load_results за допомогою @singledispatch. Окремі реєстрації обробляють рядок, Path, відкритий текстовий файл і список словників. Для файлових джерел визначається формат за розширенням: CSV читається через csv.DictReader, JSON — через json.loads. Для непідтримуваного типу генерується TypeError, а некоректна структура даних спричиняє ValueError.

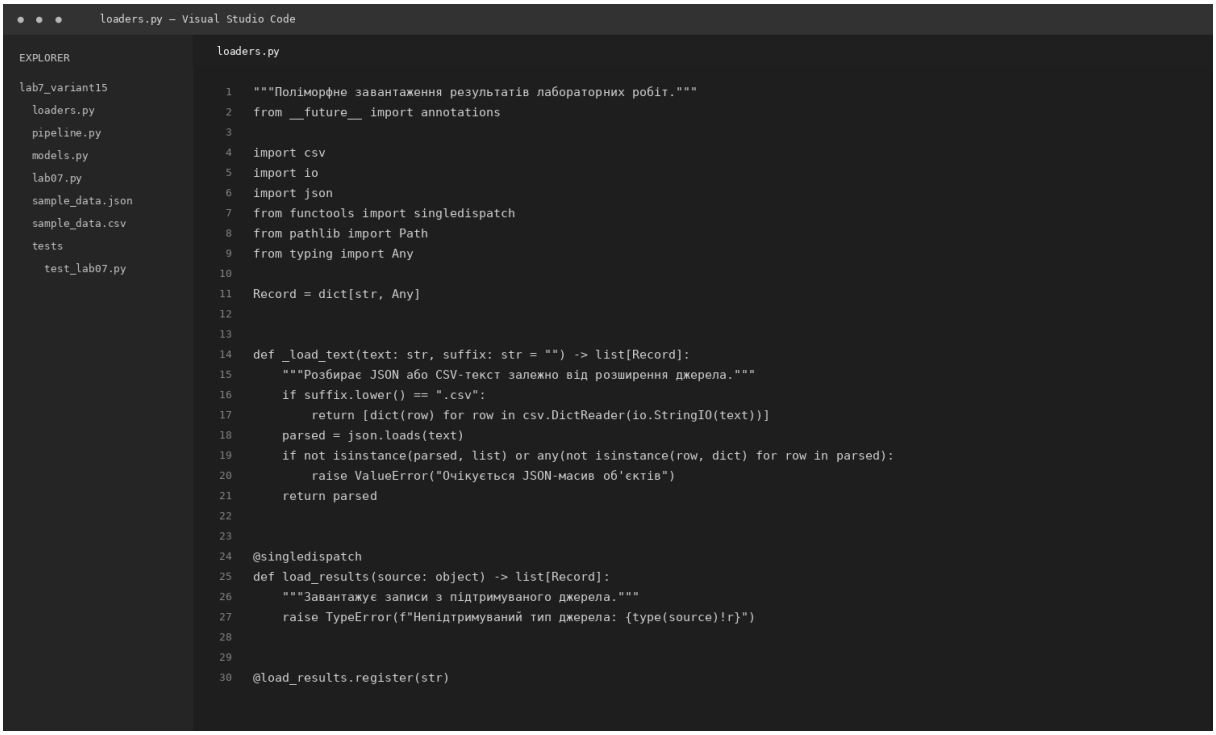


Рисунок 1 – Реалізація поліморфного завантаження даних

3. Реалізація функціонального конвеєра

У файлі pipeline.py функція pipeline реалізована через reduce. Функція clean_and_rank виконує послідовність етапів: завантаження, нормалізація,

фільтрація за submitted, обчислення final_score, сортування та вибір Top-N. Для нормалізації застосовано itemgetter і methodcaller, а partial використано для конфігурації map, filter, сортування та обмеження кількості результатів.

Підсумковий бал обчислюється за формулою: final_score = max(0, score − 0,5 × (attempts − 1)). Отже, за кожну спробу після першої віднімається 0,5 бала. Цю політику визначено в реалізації, оскільки методичні вказівки вимагають обчислити підсумковий бал, але не задають конкретної формули.

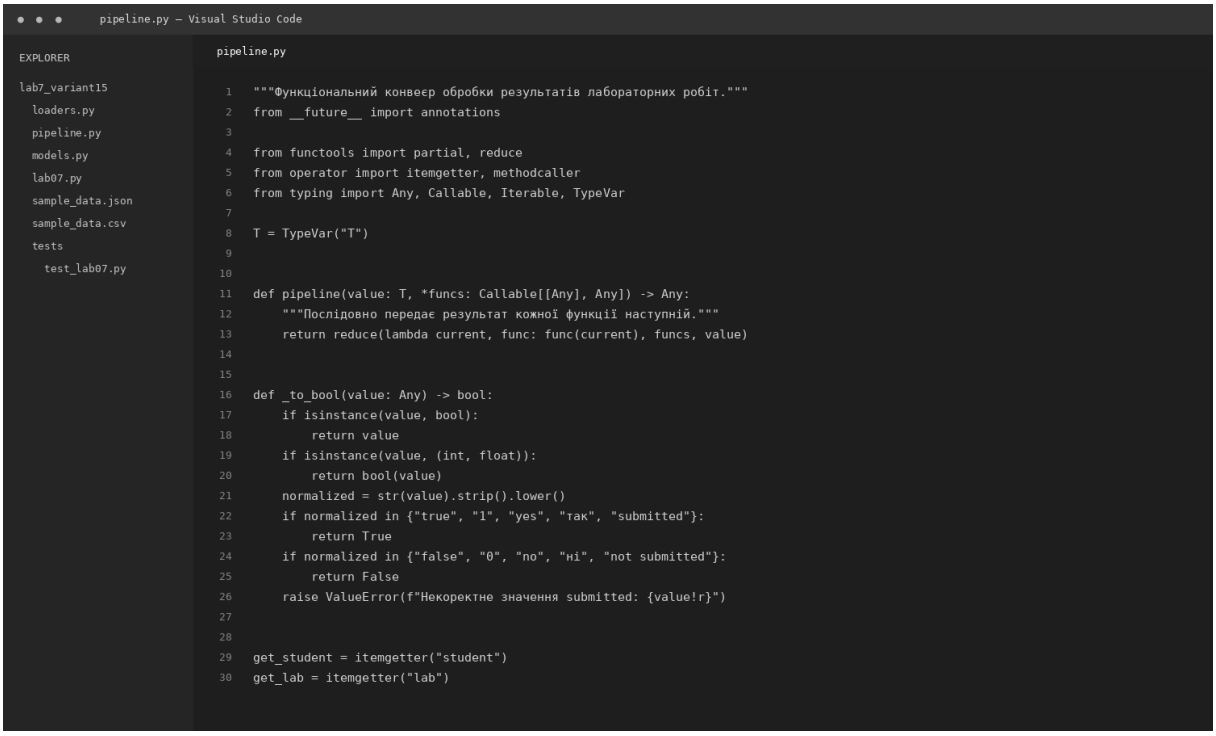


Рисунок 2 – Реалізація конвеєра обробки результатів

4. Клас LabResult та порівняння об’єктів

У файлі models.py створено клас LabResult із декоратором @total_ordering. Методи __eq__ та __lt__ порівнюють результати за кортежем (score, -attempts, student). Вищий бал дає кращий результат; за однакового бала перевагу має менша кількість спроб. Для об’єктів іншого типу методи повертають NotImplemented. Декоратор автоматично формує решту операторів порівняння.

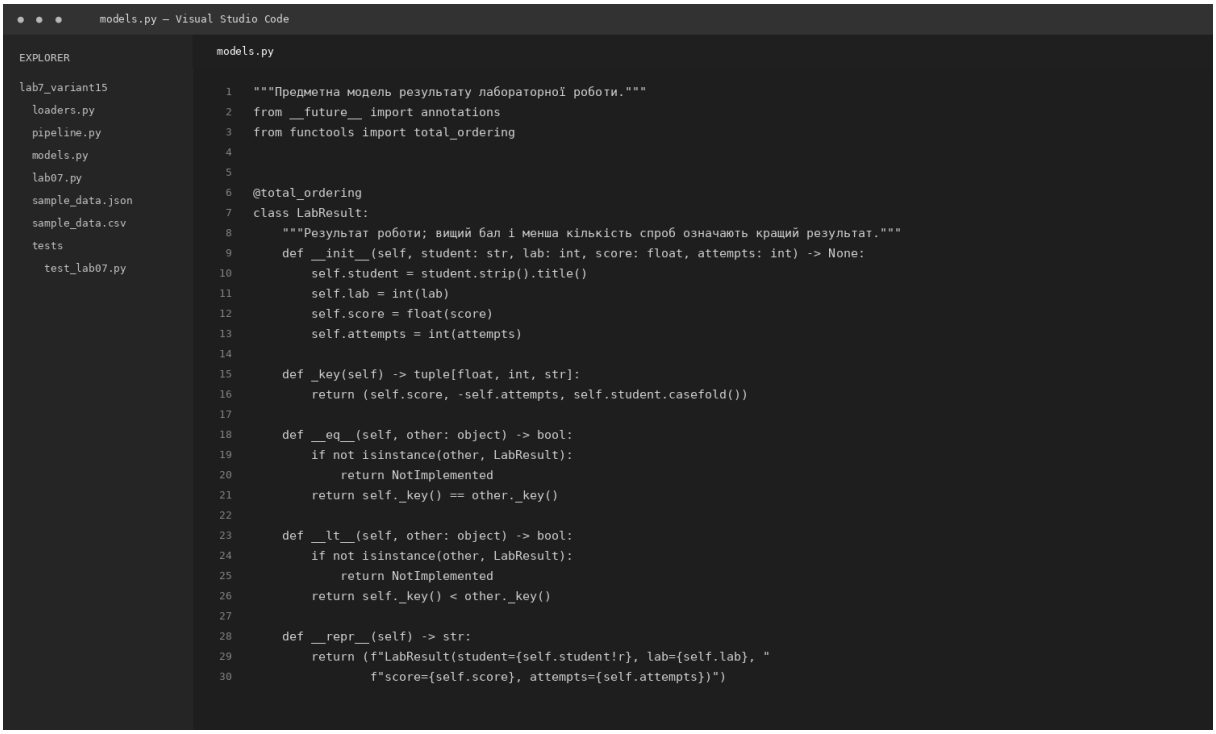


Рисунок 3 – Клас LabResult з повним порядком порівняння

5. Демонстрація роботи програми

У файлі lab07.py реалізовано запуск демонстрації. Програма читає sample_data.json, виводить три найкращі здані роботи, демонструє завантаження CSV і порівняння об’єктів LabResult. Прикладові дані містять як здані, так і нездані роботи, тому можна перевірити коректність фільтрації.

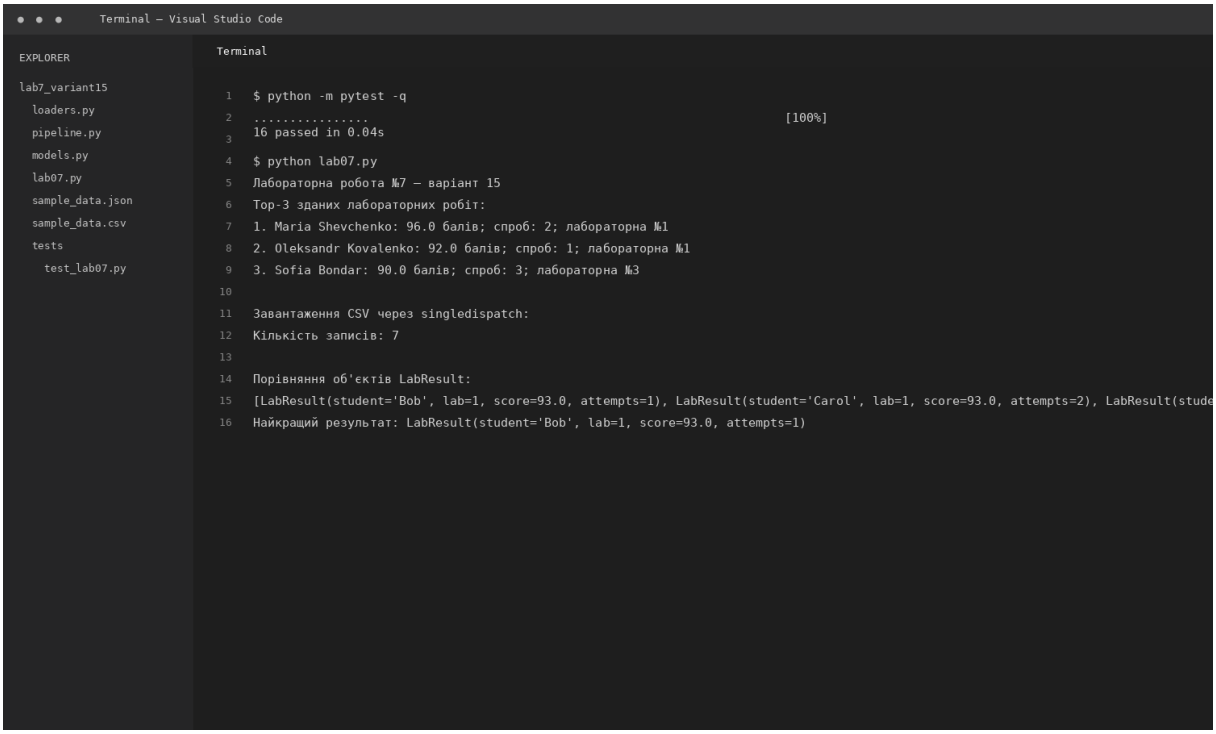


Рисунок 4 – Результат запуску програми та автоматичних тестів

6. Тестування

Я підготувала pytest-тести для pipeline, partial, усіх підтримуваних форматів load_results, нормалізації та обчислення бала, фільтрації і сортування Top-N, обробки помилок, а також операторів порівняння класу LabResult. Тести перевіряють JSON-рядок, шлях str, Path, відкритий файл, список словників і CSV.

Результат запуску pytest: ..... [100%] 16 passed in 0.05s. Усі 16 тестів завершилися успішно.

7. Перевірка якості коду

Для проєкту підготовлено конфігурації pyproject.toml і mypy.ini, а також команди перевірки black, ruff і mypy. У поточному середовищі підтверджено виконання pytest та запуск демонстраційної програми. Інструменти black, ruff і mypy не запускалися, якщо вони не встановлені в середовищі; тому їхній результат не позначаю як успішний без фактичної перевірки.

Висновок

Під час виконання лабораторної роботи я розробила програму для обробки результатів лабораторних робіт студентів відповідно до варіанта 15. Я реалізувала універсальний конвеєр pipeline на основі reduce, використала partial для налаштування етапів обробки та засоби operator для доступу до полів і перетворення рядків. Я створила поліморфну функцію load_results на основі singledispatch із підтримкою JSON, CSV, шляхів, відкритих файлів і готових списків словників. Також я реалізувала клас LabResult із total_ordering та перевірила оператори порівняння. Автоматичні тести підтвердили коректність основних функцій програми.